

1) ARCHIVO RAÍZ Y ARCHIVO PRINCIPAL

- Archivo raíz del proyecto: ``pom.xml``. Explica: es el descriptor de Maven; define el artefacto, versión, dependencias y plugins; se usa para compilar, probar y empaclar la aplicación. Al abrir el proyecto en un IDE o ejecutar ``mvn package`` Maven lee este archivo.
- Archivo principal / punto de entrada de la aplicación backend: ``src/main/java/com/unimatch/UniMatchApplication.java``. Explica: contiene el método ``main()`` que inicia Spring Boot y arranca el contenedor de la aplicación. Es el entrypoint de la aplicación Java.
- Archivo principal del frontend (SPA): ``src/main/resources/static/index.html``. Explica: es la página única servida al navegador; carga los módulos JavaScript (router y vistas) y actúa como punto de entrada para la UI.

2) RAÍZ

- ``pom.xml``: (ver arriba) configuración de construcción y dependencias (Spring Boot Web, Data JPA, H2, Oracle JDBC, Lombok, etc.). Importante para CI/CD y empaquetado.
- ``proyectos.json``: archivo estático de ejemplo que el frontend puede usar como datos de muestra; no es usado por el backend.
- ``data/unimatch.mv.db``: archivo de la base de datos H2 en desarrollo; contiene tablas y datos cuando el perfil ``h2`` está activo.

3) BACKEND (carpeta ``Backend/``)

- ``Backend/UniMatchController.java``: controlador de demostración que expone ``/api/v1/login`` para validar dominio de correo; sirve para pruebas rápidas o demos locales.

4) SPRING BOOT APP

- ``src/main/java/com/unimatch/UniMatchApplication.java``: clase con ``main()`` que arranca Spring Boot.

5) CONFIGURACIÓN (`src/main/java/com/unimatch/config`)

- ``CorsConfig.java``: registra un ``WebMvcConfigurer`` que permite CORS para rutas ``/api/**``. Propósito: permitir al frontend (ej. servido desde otro origen durante desarrollo) realizar fetch a la API. En producción conviene restringir ``allowedOrigins`` a dominios concretos.
- ``WebConfig.java``: configura ``ResourceHandler`` para servir archivos estáticos desde varias ubicaciones (carpeta raíz del proyecto y ``classpath:/static/``). Además redirige ``/`` hacia ``index.html``. Propósito: exponer la SPA y recursos (CSS/JS) por parte del backend.

6) CONTROLADORES (`src/main/java/com/unimatch/controller`)

- ``AuthController.java`` (`/api/auth``):
 - ``POST /registro``: persiste una nueva entidad ``Usuario`` en la BD si el email no existe. Devuelve un token mock y el usuario.
 - ``POST /login``: valida email + password (comparación en claro) y devuelve token mock.
 - ``GET /me``: extrae el id desde el token mock y devuelve el ``Usuario`` (simula comportamiento de endpoint 'perfil' de un JWT).
 - Propósito: gestionar cuentas y sesión (en MVP). En producción reemplazar por Spring Security y JWT/BCrypt.
- ``ChatController.java`` (`/api/chat``):
 - ``GET /conversaciones``: construye una lista de conversaciones a partir de mensajes existentes (agrupar por persona con la que hubo intercambio).
 - ``GET /mensajes/{otroId}``: devuelve la conversación entre el usuario (del token) y ``otroId`` ordenada por fecha.
 - ``POST /mensajes``: guarda un ``Mensaje`` (texto o imagen en base64) con remitente = usuario del token.
 - Propósito: manejar intercambio de mensajes; frontend hace polling para actualizar.
- ``ProjectController.java`` (`/api/proyectos``):
 - ``GET /proyectos``: lista proyectos; soporta filtros ``estado`` y ``directorId``.
 - ``GET /proyectos/{id}``: obtiene proyecto por id.
 - ``POST /proyectos``: crea un nuevo ``Project`` asignando como director al usuario autenticado; incrementa contador ``dirigidos`` del usuario.
 - ``PUT /proyectos/{id}/finalizar``: marca ``finalizado`` y guarda campos de resultado.
 - Propósito: CRUD de proyectos y flujo de publicación/finalización.
- ``SolicitudController.java`` (`/api/solicitudes``):
 - ``POST /solicitudes``: crear solicitud a proyecto; copia metadatos del solicitante y del proyecto para vista rápida.
 - ``GET /solicitudes/enviadas`` y ``/recibidas``: listar por solicitante o director.
 - ``GET /solicitudes/proyecto/{id}``: listar por proyecto y estado.

- `GET /solicitudes/check?projectId=`: chequear existencia de solicitud del usuario.
- `PUT /solicitudes/{id}`: responder (aceptar/rechazar). Si se acepta incrementa `participaciones` del usuario solicitante.
- Propósito: gestionar flujo de solicitudes (postulación, aceptación, rechazo).

7) MODELOS (src/main/java/com/unimatch/model)

- `Usuario`: representa cuentas; campos clave: `email` (único), `password` (en texto en MVP), `rol` (estudiante/maestro), `habilidades` (ElementCollection) y contadores `participaciones` y `dirigidos`.
- Propósito: identidad y perfil dentro de UniMatch.
- `Project`: representa un proyecto publicado; contiene `habilidadesReq` (lista), información redundante del director (`directorId`, `directorNombre`) pensada para evitar joins frecuentes en frontend.
- Propósito: persistir proyectos y resultados.
- `Solicitud`: representa una postulación de usuario a un proyecto; guarda información resumida del solicitante para facilidad de lectura.
- `Mensaje`: mensajes entre usuarios; puede contener `imagenBase64` (Lob). Propósito: chat P2P.
- `Evaluacion`: evaluaciones de participantes; contiene `estrellas`, `etiquetas`, `resena`.

8) REPOSITORIES (src/main/java/com/unimatch/repository)

- Interfaces que extienden `JpaRepository` por entidad y exponen consultas comunes (ej. `findByEmail`, `findByEstado`, `findByDirectorId`, `findByProjectIdAndSolicitanteId`, etc.).
- Propósito: capa de acceso a datos para los controladores.

9) RECURSOS (src/main/resources)

- `application.properties`: perfil por defecto (`h2`) y configuración H2 (archivo), JPA `ddl-auto=update` en desarrollo, límites de subida y `spring.web.resources.static-locations`.
- `application-oracle.properties`: perfil `oracle` con URL de conexión, usuario y password y `ddl-auto=none` (espera que tablas ya existan en producción).
- `static/`:
- `index.html`: SPA principal (entrada UI).
- `css/style.css`: estilos.
- `js/`:
- `firebase-config.js`: en realidad contiene `API` cliente para consumir la API REST (fetch) y utilidades; `API.BASE_URL` apunta a un ngrok público — en producción debería apuntar a la URL real y usarse variables de entorno.
- `app.js`: router SPA, gestión de sesión y control de rutas protegidas.
- `auth.js`: páginas de login/registro y validaciones frontend.
- `projects.js`: lógica del dashboard, publicar, detalles, solicitudes.
- `chat.js`: UI de chat y envío/recepción de mensajes (polling cada 3s).
- `profile.js`: vista de perfil, expediente y edición de habilidades (hace PUT a `/usuarios/{id}/habilidades`).

10) COMPORTAMIENTOS IMPORTANTES Y RAZONES

- Por qué se guarda `directorNombre` dentro de `Project`? Para mostrar rápido en frontend sin realizar join/lookup; trade-off: duplicación de datos pero consultas más simples.
- Por qué `imagenBase64` en `Mensaje`? Implementación rápida para MVP, evita infraestructura externa, pero tiene impactos: tamaño de BD, rendimiento y backups más pesados.
- Por qué token mock? Simplifica MVP (evita implementar auth completa). Riesgo: no es seguro ni escalable.

11) RECOMENDACIONES PRIORITARIAS

- Implementar Spring Security + JWT y validar tokens automáticamente (evitar parseo manual del token en cada controlador).
- Usar `BCryptPasswordEncoder` para almacenar contraseñas.
- Mover almacenamiento de binarios a un servicio de objetos (S3 / Blob Storage) y guardar sólo URL en la BD.
- Restringir CORS y parametrizar `API.BASE_URL`.

FIN: estas descripciones ahora explican el propósito y la razón de cada archivo o componente.